PYTHON + NODE.JS PROJECT ROADMAP · 2026.10

从看懂 dsh-dqd-scroll 到独立做出同类插件

给前端开发者的 10 周项目学习路线

> **从你现在卡住的调用链出发，逐步学会自己划分目录、进程、数据与插件边界。**

约 60–80 小时　　从离线 SQLite 路径到 DSH 插件　　2026-10-25

代码阅读、练习任务、毕业项目与验收标准　　01 / LEARNING PATH

摘要

你已经会 JavaScript 和 Python 的基本语法，也有前端开发经验；当前真正卡住的，是程序如何从“插件安装目录”找到 Python 包、用户数据为什么不放 .py、以及一个 JavaScript 调度脚本怎样把 CLI 子命令交给 Python。这条路线从这些具体疑问出发，目标不是再上一遍语法课，而是建立跨进程、跨目录、跨语言的项目心智模型。

建议用 **10 周、每周 6–8 小时** 完成，约 60–80 小时。先从本地 SQLite 的 daily-report 跑通“读数据—组织报告—写 Markdown”，再学习动态日报的 Python CLI 与 Node.js 子进程边界，最后在独立练习目录做一个同类 DSH 插件。全程先用样例数据和只读报告，把网络/API 与定时调度放在后段。

本报告按 dsh-dqd-scroll v0.3.0 源码结构编排，证据来自截至 2026-10-25 可读到的本地项目文件。代码阅读能说明调用关系和预期行为；它本身不能证明某台电脑上的插件安装、动态接口或定时任务已经成功运行。

学完的标志： 给你一个新需求，你能自己定位入口、画出数据流、确定改动层、写出最小垂直切片、在隔离环境验证，并解释文件落在哪里、失败在哪里。

目录

摘要 2

第一幕｜先把眼前这条调用链看明白 4

第二幕｜10 周学习路线：从读懂到能改 6

第三幕｜毕业项目：从空目录独立做出同类插件 14

工具箱｜以后遇到陌生项目就问这六个问题 15

第一幕｜先把眼前这条调用链看明白

DSH 插件刚安装好，你打开用户数据目录，没找到任何 .py 文件，于是自然会问：Python 代码是不是没装进去？接着你看到 index.js、dynamics-scheduled-runner.js 和名为 dynamics-report 的命令，便开始找一个同名的 dynamics-report.py。

这个猜测很合理，但把三个不同概念放在了一起：**代码包放置位置、用户运行数据位置、CLI 子命令名称。**先把它们分开，后面学习就会顺很多。

一张目录图：插件代码与本机数据各有职责

位置	主要内容	谁读取或写入
DSH 插件安装目录（项目的 projectDir）	index.js、Node 模块、src/dqd_scroll/**/*.py、配置模板、teams.json、提示词	DSH 加载插件；Node/Python 运行代码；Python 通过 PYTHONPATH=/src 导入包
DSH 用户数据目录（项目的 stateDir）	config.toml、.venv、解释器记录、SQLite、日志、报告、调度状态和运行入口	用户配置、Python 环境、执行结果和本机调度状态
Python 包入口	src/dqd_scroll/**main**.py → cli.py	python -m dqd_scroll ... 从这里开始，不按子命令名寻找同名 .py 文件

package.json 的发布清单包含 src/dqd_scroll/*.py*、*src/dqd_scroll/api/*.py 和 src/dqd_scroll/lib/*.py。因此 .py 文件属于插件包；initializeUserData() 的工作是初始化用户配置，并按条件迁移旧数据库和旧报告，不是把 Python 源码复制到用户数据目录。调度器会把 JavaScript runner 和时间工具复制到用户数据目录，Python 源码仍由插件目录提供。

你问的四个问题，先给出准确答案

index.js 是什么？ 它是 DSH 插件入口。DSH 加载插件时会调用 apply(ctx)；函数读取插件目录与用户数据目录、初始化用户数据、注册设置页 API，并调用 scheduler.sync()。仅加载 index.js 不代表马上启动了 Python。

index.js 执行 Python 了吗？ 插件激活路径里没有直接执行 Python。点击“准备运行环境”会建立 .venv、安装依赖，并运行 python -m dqd_scroll db-init ... 初始化数据库。这是设置 API 触发的另一条路径。

lib/dynamics-scheduled-runner.js 做什么？ 它在手动运行或定时任务触发后读取调度配置，决定执行范围，然后通过 Node 的 spawn(command, args, options) 启动命令。command 模式把 Python 解释器作为 command，把参数数组传给它，并设置工作目录与环境变量。

它怎样连到 dynamics-report.py？ 它不连到这个文件，因为当前调用链没有一个叫 dynamics-report.py 的入口。dynamics-report 是 argparse 注册的子命令：

node runner → python -m dqd_scroll dynamics-report → **main**.py → cli.py 解析子命令 → generate_dynamics_report() → workflow.build_team_dynamics_reports() → API/筛选/格式化 → Markdown 文件

Python 的 -m dqd_scroll 表示按模块运行 dqd_scroll 包；包的 **main**.py 再调用 cli.main()。CLI 用 argparse 识别 dynamics-report，并分发到对应 handler。src/dqd_scroll/lib/dynamics_report.py 负责把动态整理成 Markdown，属于格式化器，不是进程入口。

先记住这三条边界

1. **插件目录提供程序代码；用户数据目录保存本机配置、环境、状态和产物。** 判断文件在哪，先问“它是程序还是运行数据？”
2. **注册任务不等于执行任务。** scheduler.sync() 配置/同步调度；runner 被触发后才启动命令。
3. **命令、模块、文件是三种名称。** dynamics-report 是子命令，dqd_scroll 是包名，dynamics_report.py 是项目中的一个模块文件。

第一幕要做完的练习是画出上面的调用链，并在源码里为每个箭头找到一处证据。暂时不改代码，也不用启动实时动态接口。你已经掌握了正确的阅读姿势：从用户动作往下追到实际写文件的位置，再从 Python 入口反向确认命令如何分发。

第二幕｜10 周学习路线：从读懂到能改

每周都交付一个看得见的产物。做完标准不是“看完文件”，而是能不看答案讲清楚输入、处理、输出和失败位置。现有项目作为**阅读样本**；动手代码放在独立练习目录或副本中，避免把学习实验写进用户现有代码。

开始前：先搭一套固定的学习方法

把练习放在独立目录，例如 C:/www/dsh-dqd-scroll-practice；当前 dsh-dqd-scroll 仓库只读，不把练习代码直接加进去。建立一份学习笔记，每个问题都用同一行格式记录：**问题｜源码位置｜输入｜下一跳｜输出/副作用｜证据｜还不确定的部分**。不要把“我猜它这样做”记成事实。

每周按四次学习完成，合计约 6 小时：第一次 60 分钟读入口和名词；第二次 90 分钟沿函数调用追数据；第三次 2 小时在练习目录动手；第四次 90 分钟关掉源码凭记忆复述、对照笔记补漏。每周多出的 1–2 小时用来重做卡住的步骤，不用来连续通读更多文件。

读文件时先搜索符号，再读命中行附近的小段，然后跟一个调用者和一个被调用者；每次只回答一个问题。例如要查插件目录由谁设置，先搜 projectDir，再看值从哪来、哪些函数消费它。这样比一次打开几百行更容易形成调用链。

```
rg -n "projectDir|stateDir|initializeUserData|scheduler.sync" C:/www/dsh-dqd-scroll/index.js;
Get-Content C:/www/dsh-dqd-scroll/index.js | Select-Object -Skip 10 -First 45;
```

第一周先把这套笔记和目录建好。以后每次写代码前先写预期输入和输出，运行后再记录真实结果；静态读码、练习目录运行、真实 DSH 环境运行要分开标记。

第 1 周｜建立项目地图，解决当前困惑

时间： 6 小时。**目标：** 知道插件从哪加载、Python 从哪导入、结果写到哪。

阅读 package.json、index.js、lib/user-data.js、lib/scheduler.js、settings-api.js，先只看入口和函数调用，不追每个小工具。分别画出插件目录与 stateDir 的职责，再画“激活、准备环境、手动运行”三条不同路径。

练习与产物： 做一张目录责任表和一张时序图；口头回答 index.js 是否启动 Python、.py 为什么不出现在数据目录、scheduler.sync() 是否等于执行日报。

完成标准： 能准确区分 projectDir、stateDir、PYTHONPATH、工作目录 cwd，并把每个运行产物归到正确目录。

本周过程：从安装文件追到实际动作

1. **第一次：拆包清单，约 60 分钟。** 打开 package.json，只标记 main、exports、files、engines、dsh。把 files 清单中 JavaScript、Python、配置、提示词分别抄进“插件包可能携带什么”表；不要从文件名推断安装位置，先找到清单证据。
2. **第二次：插件激活，约 90 分钟。** 从 index.js 的 apply(ctx) 开始，用箭头标记 projectDir 如何从当前模块文件算出、stateDir 如何由 DSH 提供、initializeUserData 收到什么参数。接着读 lib/user-data.js，把每个 copy/mkdir 操作记成“源路径 → 目标路径 → 触发条件”。最后明确：函数没有遍历 Python 源目录，所以它不会把 .py 复制去 stateDir。

3. **第三次：两条 Python 路径，约 2 小时。** 在 settings-api.js 找到 python-environment/install 路由，顺着 uv 下载、Python 安装、.venv、依赖、db-init 记一条“准备环境”时序。再从 index.js 的 scheduler.sync 进入 lib/scheduler.js，追它写入/复制哪些 runner 文件、注册了什么调度。把“登记一个以后会触发的动作”和“现在启动日报”分成两种颜色。
4. **第四次：手动执行，约 90 分钟。** 从设置页的 run-now 请求追到 scheduler.runNow，再追到用户数据目录里的 runner，最后追 settings.pythonPath 和 Python args。画三条泳道：DSH 插件激活、准备环境、跑动态日报。箭头旁边写出“读/写文件”“启动 Node 子进程”“启动 Python 子进程”。

复盘时盖住源码，回答：index.js 何时执行、db-init 是谁启动、dynamics-report 是否文件名、动态 runner 从哪个目录导入 Python 包。答错一项就回到对应的调用点补证据，不靠再读一遍整个项目。

第 2 周｜Node.js 文件、路径和子进程

时间： 6–8 小时。**目标：** 读懂前端之外的 Node 运行时工作。

聚焦 node:fs、node:path、node:url、ESM 导入、process.env、process.argv、child_process.spawn()。重点看参数数组、cwd、环境变量、stdout/stderr、退出码和 error/close 事件。不要只把整条 shell 命令拼成字符串；数组参数更容易分辨可执行文件与参数。

练习与产物： 在练习目录写一个小 JS 脚本：接收一个路径，读取 UTF-8 文本，调用 Python 打印一行结果，再把退出码记录出来。故意传入不存在的路径，观察启动错误和非零退出码如何区分。

完成标准： 你能解释 spawn(command, args, { cwd, env }) 中三个输入分别做什么，并能从日志定位是“进程没启动”还是“进程启动后返回失败”。

本周过程：把 Node 当成文件与进程的协调者来练

1. **第一次：文件与路径，约 60 分钟。** 阅读 lib/user-data.js 和 settings-api.js 中的 readFileSync、writeFileSync、copyFileSync、mkdirSync、existsSync，以及 node:path 的 join。每遇到一个路径，就写下它相对谁：插件目录、用户目录还是当前工作目录。重点看 daily-report 的输出为什么由 Python 的 Path.cwd() 决定。
2. **第二次：模块路径与进程参数，约 90 分钟。** 阅读 index.js 的 import.meta.url/fileURLToPath/dirname，理解“当前源文件位置”和 process.cwd() 不一定相同。再读 dynamics-scheduled-runner.js 的 command、args、cwd、env。把 args 数组逐项排成命令行，解释 -m、包名、子命令和每个选项值分别对应什么。
3. **第三次：做一个文件练习，约 2 小时。** 在练习目录放一份小 JSON，写一个 Node 脚本读取 UTF-8、JSON.parse、挑两个字段，再写入练习目录的 Markdown 文件。先用绝对路径跑通，再改成以脚本目录为基准的 join；记录两种路径各自在改变工作目录时的结果。输入、输出都放在练习目录。
4. **第四次：接 Python 子进程，约 90 分钟。** 让同一个 Node 脚本用 spawn 启动一个最小 Python 命令。先只打印固定文本，再让 Python 读取传入的文件路径。逐项加入 cwd、PYTHONPATH、UTF-8 环境变量和 stdout/stderr 采集；每次只改一项，观察输出变化。用错误清单区分“路径不存在”“JSON 格式错误”“解释器没有启动”“Python 运行后返回非零码”。

完成后用一句话解释：fs 处理文件，path 组合路径，spawn 启动另一个进程；Node 不会因为一个文件叫 dynamics-scheduled-runner 就自动理解 Python 子命令。若读文件成功但 Python 找不到包，优先检查 PYTHONPATH 与解释器，不要去改 Python 函数。

第 3 周｜Python 包、-m 和命令行参数

时间： 6 小时。**目标：** 把“一个 Python 脚本”升级为“一个可导入、可分发、可执行的包”。

阅读 pyproject.toml、src/dqd_scroll/**main**.py、src/dqd_scroll/cli.py。画出 python -m dqd_scroll 的解析路径，以及 argparse 如何把 daily-report、db-init、dynamics-report 映射到处理函数。理解包目录、入口模块、子命令、参数解析是不同层次。

练习与产物： 在独立小包里建 src/hello_tool/**main**.py 和 cli.py，支持 hello --name Ada，并从终端使用 python -m hello_tool --name Ada 调用。

完成标准： 你不再根据命令名猜 Python 文件名；你能找到包入口、命令注册点、参数类型与 handler。

本周过程：沿着包入口走一遍 argparse

1. **第一次：认识安装布局，约 60 分钟。** 阅读 pyproject.toml 的 project、requires-python、dependencies、scripts 和 setuptools.packages.find。画出源码根目录 src 与包目录 src/dqd_scroll 的关系。回答 Python 要导入 dqd_scroll 时，为什么 PYTHONPATH 应指向 src，而不是包目录本身。
2. **第二次：入口到解析器，约 90 分钟。** 先读 **main**.py 的两行，再打开 cli.py 的 build_parser() 和 main()。在纸上写：解释器收到 -m dqd_scroll 后加载哪个文件；谁创建 ArgumentParser；谁创建 subparsers；set_defaults(handler=...) 把哪个函数挂到命令上。
3. **第三次：追一条具体命令，约 2 小时。** 只追 dynamics-report：从 add_parser 找到它的参数；找到 set_defaults；在 main() 找到对应分发；再进入 generate_dynamics_report。把 --config、--teams、--since、--until 分别记成“字符串/日期如何转成 Python 参数对象”，不要同时读函数里的 API 细节。
4. **第四次：自己建一个最小包，约 90 分钟。** 在练习目录建立 src/hello_tool/**main**.py、cli.py 和 pyproject.toml。先让 python -m hello_tool --help 显示帮助，再增加一个 --name 参数并打印结果。最后删掉一个参数值或输错命令，观察 argparse 怎样指出参数错误；在笔记里画出与 dqd-scroll 相同的入口图。

这一周不要尝试把 hello_tool 装成完整产品。要学会的是“包入口、CLI 命令、handler”三层。能自己添加一个子命令、说明它进入了哪个函数，才算掌握。

第 4 周｜用本地数据跑通最小 Python 业务链

时间： 6–8 小时。**目标：** 理解项目最小、可离线验证的端到端路径。

从 daily-report 开始，而不是从网络动态开始。追踪 CLI 参数 → 配置/数据库读取 → Repository 查询 → 报告格式化 → out/<日期>.md 写入。学习 Python pathlib、UTF-8 文件读写、SQLite 查询与 Markdown 输出之间的职责边界。

练习与产物： 在独立练习项目使用一份小 SQLite 样例数据，写一个只读查询和 Markdown 生成器。输入固定的日期与球队，输出一份确定的报告；重复运行时不改变数据库。

完成标准： 能从输出文件反推它由哪个函数写入，能说明查询条件和文件路径由谁决定；第一次跑通不依赖网络。

本周过程：先读懂只读日报，再亲自走数据

1. **第一次：从 CLI 定义入手，约 60 分钟。** 在 cli.py 找 daily-report 的 add_parser、--database、--team-id、--as-of 和 handler。把一个命令行参数写成一条对应关系：命令文本 → argparse 类型转换 → main() 传给哪个函数。
2. **第二次：跟数据库读取，约 90 分钟。** 读 generate_daily_report()，只跟这条路径：创建 Repository → get_team()/get_team_schedule() → formatter。然后到 repository.py 找 get_team_schedule()，看 SELECT 的 WHERE 和 ORDER BY；把“筛谁、筛哪个赛季、怎么排序”翻成中文。此处不要求记住 SQLite 语法细节。
3. **第三次：跟 Markdown 与文件位置，约 90 分钟。** 读 lib/report.py 的格式函数，标出它接收的数据和返回值。回到 cli.py 找 Path.cwd()/out/日期.md、mkdir、write_text。做一个表格：数据库里已有的信息 → formatter 如何显示 → 哪个代码行把文字写入磁盘。
4. **第四次：在练习目录复现，约 2 小时。** 使用已初始化数据库的副本；从练习目录启动只读日报命令，使输出 out/ 留在练习目录。逐段核对球队名、日期范围、赛程行数与 Markdown。若本机没有可用数据库，不要改配置去猜路径；先用小 SQLite 表建立自己的最小样例，复现 Repository 查询和 formatter。

最后把“命令执行前后哪些东西会变”列出来。这个日报读取数据库但写 Markdown；因此能解释它为何适合作为第一次端到端路径，也能避免把读取和导入混为一谈。

第 5 周｜数据模型、清洗和 SQLite 写入边界

时间： 6–8 小时。**目标：** 学会判断哪些操作只是读数据，哪些会更改数据库。

读 models.py、repository.py、workflow.py 与 lib/team_schedule.py、lib/team_roster.py。沿“响应对象 → 校验/解析 → 模型 → Repository → SQLite”追一个导入路径；再对照动态报告为何不调用 Repository、也不写入 SQLite。理解事务和“先验证、再一次性写入”的价值。

练习与产物： 给练习项目增加一条从 JSON fixture 导入 SQLite 的路径：拒绝缺少关键字段的数据；有效数据在单个事务内替换；导入后输出行数与报告。

完成标准： 能指认解析、校验、持久化分别由哪层负责，并能说明操作失败时数据库应保持什么状态。

本周过程：对照一份响应样例看数据怎样进库

1. **第一次：先看数据形状，约 60 分钟。** 阅读 fixtures/README.md 和两份 roster/schedule JSON 样例。不要通读完整 JSON；先找根对象、数组字段、ID、名称、时间和赛季字段。把“样例原字段”和 models.py 中的模型字段做成映射表。
2. **第二次：阅读校验器，约 90 分钟。** 选 team_schedule.py 的 parse_team_schedule()，沿着选当前赛季、解析单场、校验 ID、检查重复 match_id 的顺序读。再选 team_roster.py 的人员去重逻辑。每个 if 都回答“防止哪种脏数据进入下一层”；不要只抄函数注释。
3. **第三次：读 Repository 的写入，约 90 分钟。** 从 workflow.save_team_schedule_response() 追到 Repository.replace_team_schedule()。圈出事务开始点、删除旧赛程的条件、批量插入点和异常抛出点。再看 get_team_schedule() 是怎样读回数据。用图表示“解析失败时不写；验证通过后在事务里替换”。

4. **第四次：在复制数据库上导入，约 2 小时。** 复制一份练习数据库，使用 schedule-import 处理样例响应，检查导入前后的赛季、行数和比赛 ID。再用一个缺字段或重复 ID 的练习响应观察校验错误；每次重新复制数据库后再试，避免错误练习污染唯一数据。对照 CLI 输出，而不是只看退出码。

对比动态日报：它从 API 取数据、在内存筛选并写两个 Markdown 文件，不经过 Repository；名单/赛程导入会校验后更新 SQLite。能画出两条不同工作流，才说明你理解“有数据”不代表“所有数据都进数据库”。

第 6 周｜Python 与 JavaScript 的交接

时间： 6 小时。**目标：** 读懂 runner 如何将 Python CLI 当成外部进程使用。

对照 lib/dynamics-scheduled-runner.js 与 settings-api.js 的 runProcess()。逐项记下 command、参数数组、cwd、PYTHONPATH、UTF-8 环境变量、stdout/stderr 与退出码。再确认 model 模式的差异：它启动 headless DSH，由会话命令行工具运行同一条 Python CLI；不能把它误解为 Python 的另一种入口。

练习与产物： 在练习项目由 Node 启动 Python 包命令，分别测试成功、参数校验失败、解释器路径错误。将输出和退出状态整理成一张“症状 → 所在边界 → 应查看证据”表。

完成标准： 你能从一条实际 args 数组复原最终命令；能区分 Python 报错、Node 启动失败和上游 DSH 调用失败。

本周过程：从 runner 还原一次真实的进程交接

1. **第一次：读子进程封装，约 60 分钟。** 看 settings-api.js 的 runProcess() 和 dynamics-scheduled-runner.js 的 runChild()。分别标记 resolve/reject 条件、stdout/stderr 从哪里接、close 事件的 code 如何返回。先不要改实现。
2. **第二次：把参数翻译成人话，约 90 分钟。** 把 runner 中 command 和 args 原样抄进表格；把 env 中 PATH/PYTHONPATH/UTF-8 项及 cwd 单列。拼出最终形式 python.exe -m dqd_scroll dynamics-report ...，然后在 cli.py 找到每个参数的接收点。核对 command 取自 pythonPath，而不是系统里随便一个 python。
3. **第三次：做最小桥接，约 2 小时。** 在练习目录让 Node 启动自己的 Python CLI，先传一个只输出文本的命令，再传文件路径和日期参数。确认 stdout 是程序正常输出、stderr 是诊断信息、close code 是最终状态；不要把三者合成一个字符串后丢掉。
4. **第四次：做故障定位卡，约 90 分钟。** 在练习项目分别制造四种故障：不存在的解释器、错误 PYTHONPATH、Python 参数校验失败、有效命令但找错输出目录。为每种记录“启动事件有没有发生、退出码是什么、看哪段日志、先查哪一项”。再回头看 model 分支：它把同一条 Python 命令交给 headless DSH 会话去执行，进程入口因此变了，Python CLI 语义没有变。

复盘时从源头按顺序读，不要先看 Python traceback 猜 Node：命令配置 → Node spawn → Python 包发现 → argparse → handler → 文件写入。定位顺序与调用顺序一致，通常能少走很多弯路。

第 7 周｜DSH 插件生命周期与设置页 API

时间： 6–8 小时。**目标：** 知道宿主怎样加载插件，UI 设置怎样抵达后端。

阅读 index.js 的 name、inject、apply(ctx)、ctx.effect()，再追 settings-api.js 的路由注册、配置读写和环境准备动作。将“设置页点击按钮”画成浏览器 → DSH API → Node handler → Python CLI 的流程。把插件入口、宿主注入、设置 API 与 Python 业务层分开记。

练习与产物： 在隔离的练习插件中注册一个只返回状态的 API；另加一个配置读写动作，确保输入经过校验后再保存。暂不接外网，不改现有插件。

完成标准： 能回答哪个组件持有 UI、哪个组件注册 API、配置文件由谁读写，以及环境初始化在什么时机发生。

本周过程：从设置页点击一路追到后端

1. **第一次：读客户端动作，约 60 分钟。** 在 client.js 找 request()、getSettings()、saveSettings()、runNow()、installPythonEnvironment()。为每个动作写 HTTP 方法、URL、body；再找 Panel 中哪些按钮调用它、响应后更新了哪块状态。先把它当作普通浏览器前端阅读。
2. **第二次：对齐服务端路由，约 90 分钟。** 在 settings-api.js 找对应的 GET/POST 路由，做一张“前端动作 ↔ 后端 URL ↔ 校验函数 ↔ 写入对象 ↔ 返回 JSON”表。特别跟踪 /run-now 和 /python-environment/install，它们会进入不同处理路径。
3. **第三次：读插件生命周期，约 90 分钟。** 回到 index.js，读 name、inject、apply(ctx)、ctx.effect()；再读 package.json 的 DSH 元数据与 cordis.patch.yml。画“DSH 什么时候加载插件，何时注册 API，何时释放定时器”。区分 DSH 提供的能力与插件自己创建的函数。
4. **第四次：自己画点击时序，约 2 小时。** 分别画“准备运行环境”和“现在跑一次”的时序图。写清每一步是浏览器 fetch、Node handler、文件读写、环境安装、scheduler 方法还是 Python spawn。最后在练习目录写一个最小 API 请求/响应合同；暂时不必把 UI 接进 DSH，先讲清协议。

这一周可以把前端经验迁移过来：client.js 是消费状态和调用接口的一端；settings-api.js 是 Node 服务端路由；Python CLI 是后端任务进程。不能因为它们在一个插件里，就把它们视为同一个运行时。

第 8 周｜配置、API 合同和动态日报

时间： 6–8 小时。**目标：** 从静态 fixture 过渡到外部数据契约。

先读 docs/api-contract.md、fixtures/、src/dqd_scroll/api/dqd_api.py、workflow.py 和 lib/dynamics_report.py。将响应结构、请求参数、分页边界、时间范围、解析规则和输出字段写成合同表。当前仓库的 fixture 样例是名单与赛程响应；没有保存动态接口原始响应，所以不要把名单/赛程样例当成动态日报的回放数据。动态日报是只读 API 流程，报告结果落盘；不要把它和 SQLite 导入流程混为一谈。

练习与产物： 用现有 roster/schedule fixture 学会“保存的响应如何离线导入”；对动态纯处理逻辑，自己在练习目录构造少量 synthetic TeamDynamic 记录，验证兴趣词筛选与 Markdown 格式化。缺字段、空列表、重复 next URL、分页上限和时间范围分别写出预期行为。实时请求是单独的验证步骤，不作为本周本地练习的前置条件。

完成标准： 你能从 API 响应追到字段校验和最终 Markdown，并能说明哪些结论来自源码/自建样例、哪些必须靠实时运行证明。

本周过程：把“代码实现”和“接口已核实”分开

1. **第一次：做证据等级表，约 60 分钟。** 阅读 docs/api-contract.md 开头的状态说明和动态接口章节。把每项写成已由请求观察核实、代码已实现但仍待验证、只有旧线索三类。不要把名单/赛程 Postman URL 和动态接口核验混在一个结论里。
2. **第二次：读客户端和分页，约 90 分钟。** 从 api/client.py 找实际 HTTP 请求如何变成 JSON；再读 api/dqd_api.py 的 fetch_team_dynamics()、第一页 URL、next URL 校验、after 时间和停止条件。每个边界都写下“什么输入会继续请求，什么条件会停止，什么情况会直接报错”。
3. **第三次：读数据模型到输出，约 90 分钟。** 在 models.py 找 TeamDynamic 与 TeamDynamicsFetch；读 lib/team_dynamics.py 的校验/兴趣筛选，再读 lib/dynamics_report.py 的两个 formatter，最后回到 workflow.build_team_dynamics_reports()。画出动态只在内存处理并产生 all/interests 两份 Markdown 的链路。
4. **第四次：做离线样例与核验计划，约 2 小时。** 手工构造最小 TeamDynamic 列表，分别包含命中词、不命中词、空标题或可选描述；运行自己写的纯函数练习，核对筛选结果和 Markdown。另写一份实时接口核验清单：请求 URL、状态码、业务 code、next URL、分页停止依据和生成文件。清单没有执行记录之前，结论仍标为未验证。

如果一个功能依赖网络，先把它拆成“纯处理逻辑”和“外部接口读取”。前者可用自建数据练习，后者要单独留下运行证据；这能让你在接口不可用时继续学习业务层。

第 9 周｜环境准备、日志、调度和发布

时间： 6–8 小时。**目标：** 理解开发机能运行与用户安装后能运行之间的差别。

阅读 settings-api.js 中 uv、Python 3.11、.venv、依赖安装和 db-init 的顺序；阅读 lib/scheduler.js 中 runner 写入、任务同步、手动运行与清理；查看 package.json 的 files 清单如何决定发布包内容。留意插件版本、Python 项目版本与 CLI --version 来自不同声明处，新增功能前要确认实际发布版本口径。

练习与产物： 写一页“干净机器安装检查单”：插件包中有什么、用户数据中会新增什么、初始化先后顺序是什么、失败看哪个日志/状态文件、卸载或重建环境会影响什么。

完成标准： 能从发布清单判断某个文件是否可能进入 npm 包；能解释调度注册和任务实际运行的区别；能给新电脑安排可复现的初始化顺序。

本周过程：分开学习安装、调度、诊断与打包

1. **第一次：环境准备，约 90 分钟。** 在 settings-api.js 从“安装环境”路由向下追：ensureUv → 安装 Python 3.11 → 创建 stateDir/.venv → 用 pyproject.toml 安装依赖 → 验证时区/版本 → 执行 db-init → 写 python-environment.json。每一步写出前置文件、命令、输出文件；解释 Python 源码为何仍由插件目录的 PYTHONPATH 提供。
2. **第二次：调度注册，约 90 分钟。** 阅读 scheduler.sync() 的公共步骤，再分别标记 Windows 任务计划和 macOS 插件计时器的分支。找出 runner、时间模块、任务配置和状态文件分别写到哪里。回答“同步设置时发生了什么”和“到时间时发生了什么”，不要把两个时刻混成一次运行。
3. **第三次：故障定位，约 90 分钟。** 读 lib/logger.js 和 runner 的状态更新，画出 running → succeeded/failed 的状态变化。把环境安装失败、Node 子进程启动失败、Python 返回非零、文件写入失败分别映射到错误日

志/退出码/状态字段。学习时只在练习环境制造错误。

4. **第四次：打包复现，约 90 分钟。** 重新读 package.json 的 files 与 engines；挑一个 .py、一个 JS runner、一个 docs 文件，判断它们是否进入 npm 包并找证据。把现有 tests/release-metadata.test.mjs、tests/user-data.test.mjs 当作行为说明来读，记录它们检查哪些约束；这一阶段以读懂断言为主，不要改源项目。最后写一份“干净机器复现步骤”，从安装插件到生成第一份 Markdown 顺序列清楚。

特别记住：插件 package version、pyproject 版本和 CLI --version 字符串可能来自不同文件。学习时把声明位置都记下，改功能时不要只更新一个版本号就认为发布信息一致。

第 10 周｜把一个小功能从需求做到交付

时间： 8 小时。**目标：** 练习自主分层，而不是照着现有文件机械复制。

在独立练习插件中新增一项“按配置关注球队生成只读 Markdown 摘要”的能力。先用本地 fixture，再通过 Python CLI 处理；Node 层负责启动命令和把状态呈现给 DSH；设置 API 只保存用户配置。写下输入、输出、文件归属、错误分支和最小验收方式后再动手。

练习与产物： 提交一份垂直切片：可读配置 → CLI 子命令 → 解析/业务函数 → Markdown 文件 → Node 手动触发入口 → 日志/退出码 → 使用说明。安排自己从空目录复现一次安装和运行。

完成标准： 别人只按你的文档操作，就能在隔离环境从初始化跑到生成报告；你能解释每个文件为什么放在该层，并能指出还没验证的边界。

本周过程：四个小里程碑，逐层加上宿主能力

1. **第一次：写合同与空项目，约 90 分钟。** 给练习插件写一页需求卡：配置字段、样例输入、筛选规则、命令格式、报告路径、错误信息、哪些能力暂不做。创建空仓库目录和 README，只建文件夹，不复制 dsh-dqd-scroll 的整份代码。
2. **第二次：独立完成 Python 路径，约 2 小时。** 先创建模型和纯处理函数，用固定 fixture 生成 Markdown；再加 argparse 子命令；最后加文件输出。每一步都从终端独立运行。把输出和输入逐项对照，确保 Python 业务没有依赖 DSH UI 或 Node 才能运行。
3. **第三次：加 Node 与设置接口，约 2 小时。** 用 Node 读取用户配置，按参数数组启动 Python；明确解释器路径、cwd、PYTHONPATH 和错误输出。再注册最小的 DSH 设置 API 和手动运行动作。先手动触发，不自动排程；一个边界稳定后再增加下一个边界。
4. **第四次：按陌生用户视角复现，约 2 小时。** 清理练习环境或换一个新目录，严格按 README 重建；检查配置、venv、数据库/样例、日志和输出分别落在哪里。记录一次成功运行和一次预期失败的证据。若必须凭记忆补步骤，README 还不完整。

交付前请让自己从空白终端回答：第一条命令是什么、Python 包如何被找到、哪个用户动作触发进程、报告输出在哪、出现非零退出码先看哪里。回答不出来的地方就是下一轮要补的文档或结构。

第三幕｜毕业项目：从空目录独立做出同类插件

题目与边界

做一个小型、只读的 DSH 插件，例如“比赛备忘日报”。它读取本地 JSON fixture 或 SQLite，按配置筛选条目，输出 Markdown；用户可在设置页选择关注项，能手动运行一次。先做 command 模式，不接模型、不排自动任务，也不把实时网络请求当作毕业门槛。等本地链路稳定后，再把 API 和调度作为后续扩展。

项目目标不是复刻懂球帝功能，而是复现它的工程边界：DSH 插件入口 + Node 文件/进程操作 + Python CLI + 数据处理 + 稳定的用户数据目录 + 可说明的打包结构。

建议的实现顺序

1. **写一页合同。** 定义设置项、fixture 字段、CLI 输入、报告路径和失败时的可观察结果。
2. **先完成 Python 独立运行。** 从 python -m report ... 生成确定的 Markdown 文件。
3. **把数据访问隔离。** fixture/SQLite/API 的加载留在数据访问层；报告逻辑接收已验证的数据。
4. **再接 Node。** 使用绝对解释器路径或明确环境配置，以参数数组启动 Python；传好 cwd 与环境变量，采集输出与退出码。
5. **最后接 DSH。** 加入口、设置 API、用户数据初始化和手动运行按钮；确认插件激活不会意外触发耗时任务。
6. **打包并复现。** 检查 package.json 发布清单，删除练习环境后按文档重建一次。

验收清单

- 新电脑能按文档准备运行环境，用户配置与输出都落在用户数据目录。
- Python 命令可脱离 DSH 单独运行；相同输入生成相同报告。
- Node 以参数数组启动 CLI，错误信息、退出码和工作目录都可追踪。
- 缺字段或无效配置会给出具体错误，不会静默生成貌似成功的空报告。
- 调度、实时 API、模型模式等未实现能力被明确标出，不在说明中写成已完成。
- 发布包清单包含运行所需文件；用户数据不需要承载 Python 源码副本。

往现有项目增加功能时，按这个顺序做决策

新功能先落到当前最靠近它的层：命令参数变化归 CLI；字段清洗归解析器；查询归 Repository；业务组合归 workflow；外部响应归 API client；输出格式归 formatter；用户触发、配置持久化和任务生命周期归插件层。跨层需求可以改多处，但每一处都要有明确的数据合同，不要把所有逻辑堆进 index.js 或一个巨大的 Python 函数。

推荐扩展梯子：**本地 Markdown 输出 → SQLite 只读查询 → fixture 导入 → API 读取 → 设置页配置 → 手动运行入口 → 日志/调度 → 跨平台安装复现。** 每一级都能独立交付和验证；做到上一级再加下一级。

工具箱｜以后遇到陌生项目就问这六个问题

开始改之前

1. **从用户动作开始，调用链在哪里？** 入口是谁，函数如何分发，最终效果落在哪个文件或 API？
2. **程序和数据分别放在哪？** 哪个是安装目录，哪个是运行数据，路径由谁提供？
3. **这次功能需要改变哪一层？** CLI、业务函数、数据层、宿主 API、UI，还是发布配置？
4. **输入与输出的合同是什么？** 字段、时间范围、文件名、编码和失败语义有没有写清？
5. **最小可验证路径是什么？** 能否先用 fixture 或本地 SQLite 证明核心逻辑，再考虑网络与调度？
6. **什么证据能证明已经完成？** 静态检查只证明代码形状；fixture/命令证明本地路径；真实 API、定时器和 DSH UI 分别需要各自的运行证据。

看到这些迹象时先停下来核对

看到	先核对
数据目录里没有 .py	发布包清单是否包含源码？运行时 PYTHONPATH 指向哪里？
找不到 dynamics-report.py	这是 CLI 子命令还是脚本文件？包入口和 handler 在哪里？
插件加载时没日报	当前只是注册调度还是已经触发 runner？执行入口由哪个动作调用？
Python 命令能手工运行、插件里失败	Node 传入的解释器、cwd、PYTHONPATH、环境变量是否相同？
报告没写到预期位置	输出路径基于当前目录、配置目录还是用户数据目录？
静态检查通过	哪些结论仍需要真实 CLI、外部 API、调度器或 DSH UI 验证？

每周都用同一张检查卡

检查项	写下你的答案
输入与处理者是谁？	用户配置、CLI 参数、SQLite 数据或 API 响应由哪一层接收？
输出与失败在哪里？	返回值、日志、SQLite、用户数据目录或 Markdown 文件；错误怎样可见？
什么证据能证明它？	源码位置、fixture 回放、命令输出、生成文件或真实运行记录

这条路线最核心的工作习惯只有一句：**顺着一条具体输入，一直追到一个具体产物。** 追通一条之后，再扩展到下一层；每加一个功能，都重新确认数据从哪里来、经过谁、最后写到哪里。

阅读地图与证据边界

需要回到源码核对时，按“宿主入口 → 用户目录 → 命令执行 → Python 分发 → 业务处理 → 输出”的顺序读。每读一个文件，只记三件事：它从哪接收输入、把工作交给谁、最终产生什么副作用。

本路线以 C:/www/dsh-dqd-scroll 的本地源码为主，重点阅读：

- 插件元数据与打包：package.json、pyproject.toml
- 插件入口与用户目录：index.js、lib/user-data.js
- 环境与设置 API：settings-api.js
- 调度与跨语言执行：lib/scheduler.js、lib/dynamics-scheduled-runner.js
- Python 入口与业务：src/dqd_scroll/**main**.py、src/dqd_scroll/cli.py、src/dqd_scroll/workflow.py
- 数据与输出：src/dqd_scroll/repository.py、src/dqd_scroll/api/dqd_api.py、src/dqd_scroll/lib/dynamics_report.py
- API 说明与离线样例：docs/api-contract.md、fixtures/

证据边界： 本路线描述的是源码中可见的结构和命令关系。它不声称当前设备上已经完成环境安装、用户界面操作、动态接口请求或定时任务验证；这些应作为学习过程中的独立验收项。源码可以证明 runner 如何构造命令，只有真实运行记录能证明本机上的命令实际成功。